

Laboratorio 09 · Migración rolling a TLS y autenticación x.509

Duración: 40 minutos · Carpeta de trabajo: entorno/

Objetivos

Vas a cifrar todo el tráfico de un replica set **que está en servicio**, sin parar la aplicación, pasando por los tres modos de TLS (`allowTLS` → `preferTLS` → `requireTLS`). Después crearás un usuario que se autentica con un certificado de cliente x.509 en lugar de con contraseña.

Contexto

Si un replica set pasara directamente de "sin TLS" a `requireTLS` , el primer nodo reiniciado dejaría de entenderse con los demás. La migración se hace en tres fases:

Modo	Conexiones entrantes	Conexiones salientes a otros miembros
<code>allowTLS</code>	Acepta TLS y sin TLS	Sin TLS
<code>preferTLS</code>	Acepta TLS y sin TLS	Con TLS
<code>requireTLS</code>	Solo TLS	Con TLS

Entre fase y fase, los clientes se van cambiando a conexiones TLS.

Paso 1 · Generar los certificados

```
cd entorno
docker compose run --rm certgen
```

El script crea una CA privada del curso, un certificado por nodo (con `mongo1` , `mongo2` o `mongo3` en el *Subject Alternative Name*) y un certificado de cliente. Inspecciona el resultado desde el toolbox:

```
docker compose exec toolbox bash
ls -l /certs
cat /certs/client-subject.txt
exit
```

Abre `entorno/compose.tls.yml` y fíjate en qué parámetros añade a cada nodo.

Paso 2 · Fase 1: reinicio rolling en `allowTLS`

En una segunda terminal, dentro del toolbox, deja corriendo el escritor para comprobar que la aplicación no se entera:

```
docker compose exec toolbox bash
mongosh "$RS" --quiet /scripts/escritor.js
```

En la primera terminal, averigua quién es el primario y reinicia los nodos con la configuración TLS **empezando por los secundarios**. Durante todo este laboratorio usa siempre los dos ficheros `-f`:

```
docker compose exec toolbox bash /scripts/primario.sh
docker compose -f compose.yml -f compose.tls.yml up -d mongo3      # un
secundario
docker compose exec toolbox bash /scripts/estado.sh                #
espera a SECONDARY y lag 0
docker compose -f compose.yml -f compose.tls.yml up -d mongo2      # el
otro secundario
docker compose exec toolbox bash /scripts/estado.sh
```

Para el primario, primero `rs.stepDown()` y después el reinicio:

```
docker compose exec toolbox bash /scripts/mongosh-rs.sh --quiet --eval
"rs.stepDown()"
docker compose -f compose.yml -f compose.tls.yml up -d mongo1      # el
que era primario
docker compose exec toolbox bash /scripts/estado.sh
```

(Si el primario no era `mongo1`, ajusta los nombres en consecuencia.)

Paso 3 · Comprobar que se aceptan ambos tipos de conexión

Dentro del toolbox:

```
mongosh "$RS" --quiet --eval "db.hello().primary"
mongosh "$RS&tls=true&tlsCAFile=/certs/ca.pem" --quiet --eval
"db.hello().primary"
mongosh "$RS&tls=true&tlsCAFile=/certs/ca.pem" --quiet --eval
"db.serverStatus().security"
```

La sección `security` de `serverStatus` muestra el subject del certificado del servidor y su fecha de caducidad: un dato que conviene vigilar con una alerta.

Paso 4 · Fases 2 y 3: `preferTLS` y `requireTLS` en caliente

El modo TLS se puede cambiar sin reiniciar con `setParameter`, nodo a nodo. Abre `mongosh` con TLS:

```
mongosh "$RS&tls=true&tlsCAFile=/certs/ca.pem"
```

```
> function modoTLS(modos) {
  ["mongo1", "mongo2", "mongo3"].forEach(h => {
    const c = new Mongo("mongodb://admin:CursoMongo2026@" + h +
":27017/?
authSource=admin&directConnection=true&tls=true&tlsCAFile=/certs/ca.pem"
);
    const r = c.getDB("admin").runCommand({ setParameter: 1, tlsMode:
modos });
    print(h + " -> " + modos + " (antes: " + r.was + ")");
  });
}
> modoTLS("preferTLS")
```

En este punto los miembros ya hablan entre sí con TLS. Es el momento de cambiar las cadenas de conexión de todas las aplicaciones para que usen `tls=true`. Cuando todas lo hagan:

```
> modoTLS("requireTLS")
> exit
```

Comprueba el resultado desde el toolbox:

```
mongosh "$RS" --quiet --eval "db.hello().primary"
mongosh "$RS&tls=true&tlsCAFile=/certs/ca.pem" --quiet --eval
"db.hello().primary"
```

El escritor de la otra terminal, que se conectó sin TLS, habrá empezado a fallar: justo lo que le pasaría a una aplicación que no se ha migrado a tiempo. Páralo con Ctrl+C.

Paso 5 · Hacer el cambio permanente

Un `setParameter` en caliente se pierde al reiniciar. Edita `entorno/.env`, cambia a `TLS_MODE=requireTLS` y aplica el cambio nodo a nodo con el mismo orden de antes (secundarios primero, `stepDown`, primario):

```
docker compose -f compose.yml -f compose.tls.yml up -d mongo3
```

Ahora todas las conexiones necesitan TLS, así que usa las variantes `-tls` de los scripts de ayuda para comprobar el estado y para hacer el `stepDown` del primario:

```
docker compose exec toolbox bash /scripts/estado-tls.sh
docker compose exec toolbox bash /scripts/mongosh-rs-tls.sh --quiet --
eval "rs.stepDown()"
```

Paso 6 · Autenticación x.509

Crea un usuario cuyo nombre es exactamente el *subject* del certificado de cliente, en la base de datos virtual `$external`:

```
docker compose exec toolbox bash
cat /certs/client-subject.txt
mongosh "$RS&tls=true&tlsCAFile=/certs/ca.pem"
```

```
> db.getSiblingDB("$external").runCommand({
  createUser: "CN=appx509,OU=Clientes,O=CursoMongo",
  roles: [ { role: "read", db: "tienda" } ]
})
> exit
```

Conéctate con el certificado, sin contraseña. Fíjate en que `$external` se escribe `%24external` en la URI:

```
X="mongodb://mongo1:27017,mongo2:27017,mongo3:27017/tienda?
replicaSet=rs0&tls=true&tlsCAFile=/certs/ca.pem&tlsCertificateKeyFile=/c
erts/client.pem&authMechanism=MONGODB-X509&authSource=%24external"
mongosh "$X" --quiet --eval "printjson(db.runCommand({ connectionStatus:
1 })).authInfo); db.pedidos.countDocuments()"
mongosh "$X" --quiet --eval "db.pedidos.insertOne({ x509: true })"
```

▼ Ver solución y explicación

`connectionStatus` muestra el usuario `CN=appx509,OU=Clientes,O=CursoMongo` autenticado en `$external` con el rol `read` sobre `tienda`, así que puede contar pedidos pero no insertar. La identidad la aporta el certificado: quien tenga `client.pem` es ese usuario, por eso la clave privada del cliente hay que protegerla como una contraseña. El *subject* del certificado de cliente debe diferir del de los miembros del cluster en al menos uno de los atributos `O`, `OU` o `DC` (aquí `OU=Clientes` frente a `OU=Servidores`); si

coincidiera, MongoDB lo trataría como un miembro del cluster. Los miembros podrían autenticarse también entre sí con x.509 (`clusterAuthMode: x509`) en lugar de con el keyfile: es lo recomendable en producción, porque un certificado se puede revocar y caduca, y un keyfile no.

Paso 7 · Volver al entorno sin TLS (¡importante para el día 4!)

Los laboratorios del día 4 trabajan sin TLS para simplificar. Edita `entorno/.env`, vuelve a poner `TLS_MODE=allowTLS` y recrea los tres nodos a la vez **sin** el override:

```
docker compose up -d
docker compose exec toolbox bash /scripts/estado.sh
```

Si algún nodo no arranca o no se ve con los demás, espera unos segundos y vuelve a mirar el estado: los tres se reinician a la vez y tardan un poco en volver a elegir primario.